# BPMN 审批流生成器

用一句自然语言，生成可直接部署的 BPMN 2.0 审批流

参赛作品介绍文档

适用于政务 / 企业审批流程的低代码配置工具

| 项目 | 内容 |
| --- | --- |
| 项目名称 | bpmn-gen —— AI 驱动的审批流生成器 |
| 技术栈 | Node.js（生成核心，零运行时依赖）+ Java（验证工具 / Flowable 引擎）+ bpmn-js（可视化） |
| 大模型 | DeepSeek（OpenAI 兼容接口），可一键切换通义千问 |
| 引擎方言 | Flowable 7（xmlns:flowable） |
| 开源许可 | Apache License 2.0 |
| 代码仓库 | https://github.com/Alter2007/bpmn-gen |
| 演示视频 | https://www.bilibili.com/video/BV1FvHA6YEyP/（P1 走大模型 / P2 离线模式） |
| 文档版本 | v1.0 |

本项目的全部技术结论均可在仓库内用一条命令复现：verify-all.cmd

# 一、项目背景：审批流配置的三重门槛

在政务与企业场景里，审批流程的配置长期是“业务说得清、开发画不准”的地带。一个典型的需求是这样的：

*“我要做一个差旅报销审批流，金额小于 1000 直接主管批，大于 1000 需要总监批，超过 5000 需要财务复核。”*

这句话对业务人员毫无歧义，但要落地成系统，中间横着三道门槛：

| 门槛 | 具体表现 | 传统代价 |
| --- | --- | --- |
| 沟通门槛 | 业务描述是自然语言，开发要实现的是流程定义，中间靠文档和会议来回对齐 | 一轮需求澄清 2~5 天，改一次再走一轮 |
| 建模门槛 | BPMN 2.0 是 OMG 规范，涉及排他/并行网关、默认流、条件表达式、图形坐标（DI） | 业务人员学不会；开发人员也要查规范 |
| 验证门槛 | 生成的流程文件是否合法、能否被引擎接受，往往要部署到引擎才知道 | 上线前才发现流程卡死 |

大模型出现后，一个非常自然的想法是：让模型直接输出 BPMN XML。

本项目在实现过程中验证了这条路的真实风险，并给出了替代方案——这也构成了本作品最主要的技术主张。

# 二、产品定位与使用方式

**一句话定位：把业务人员的一句自然语言，变成 Flowable 引擎能直接部署的 BPMN 2.0 文件。**

## 2.1 三条使用路径

| 路径 | 输入 | 是否需要大模型 | 适用场景 |
| --- | --- | --- | --- |
| 离线模板匹配 | 业务描述一句话 | 否 | 演示、断网环境、成本敏感场景 |

| 路径 | 输入 | 是否需要大模型 | 适用场景 |
| --- | --- | --- | --- |
| 大模型选模板 | 业务描述一句话 | 是（只输出 JSON） | 描述更自由、口语化的真实业务输入 |
| 直接给 IR | 结构化 JSON（中间表示） | 否 | 系统集成、批量生成、二次开发 |

以上三种能力都已在本地 Web 应用里打通：浏览器内输入一句话 → 生成 → 可视化 → 下载 .bpmn，全程不需要切到命令行。应用只监听本机回环地址，关闭窗口即结束。

### 2.2 系统做什么

管线固定为五段，每一步的职责边界清晰，且不可跳过：

| 阶段 | 职责 | 失败时的行为 |
| --- | --- | --- |
| ① 自然语言 → IR | 匹配模板并填充槽位（模型只输出 templateId + slots） | 无匹配即明确拒绝，不硬凑 |
| ② 确定性修复 | 只修确定性错误：缺 id、重复 id、自环、悬空端点、多余默认流 | 修不了的语义错误直接报错 |
| ③ 三层静态校验 | 契约 / 图论 / XML 良构 | 任一层不过即终止，不输出半成品 |
| ④ 序列化 | IR → BPMN XML，含完整 DI（图形坐标） | 序列化器不接受非 IR 输入 |
| ⑤ 独立验证 | 官方 XSD / 第三方 moddle / Flowable 引擎校验 | 报告差异，不回写产物 |

## 三、技术架构

### 3.1 分层结构

| 层 | 核心文件 | 产出 |
| --- | --- | --- |
| 交互层 | bin/nl2bpmn.js、bin/ai2bpmn.js、web/（可视化） | 命令行 / 浏览器界面 |

| 层 | 核心文件 | 产出 |
| --- | --- | --- |
| 语义层 | src/nl.js（模板匹配 + 槽位）、src/prompt.js（提示词 + 输出解析） | IR（JSON） |
| 修复层 | src/repair.js | 修好的 IR + 修复动作清单 |
| 校验层 | src/ir.js（契约）、src/graph.js（图论）、src/xml-lint.js（良构） | 结构化错误列表 |
| 序列化层 | src/layout.js（分层布局）、src/serialize.js | BPMN 2.0 XML（含 DI） |
| 验证层 | tools/xsd-validate（官方 XSD）、web/check-model.js（moddle）、tools/flowable-check（引擎） | 三层独立验证结论 |
| 编排 | src/pipeline.js | 把上面串成不可跳过的链 |

### 3.2 六层校验漏斗

这是本作品可靠性设计的核心：错误越早发现，修复成本越低。

| 层 | 检查内容 | 执行者 | 实测结果 |
| --- | --- | --- | --- |
| L1 良构 | 标签配对、根元素唯一、CDATA 完整 | 自研 xml-lint | 通过 |
| L0 契约 | 字段类型、id 唯一、自环、默认流唯一 | src/ir.js | 通过 |
| L3 图论 | 起止唯一、悬挂节点、不可达、死路、网关默认流、并行网关不带条件 | src/graph.js | 通过 |
| L2 规范 | 是否符合 BPMN 2.0 官方 schema | OMG XSD + JDK 17 校验器 | 16 / 16 文件 |
| L2b 模型 | 模型能否被标准库建出来 | 第三方 bpmn-moddle | 16 / 16 文件 |

| 层 | 检查内容 | 执行者 | 实测结果 |
| --- | --- | --- | --- |
| L4-lite 引擎 | 引擎能否解析并通过流程校验 | Flowable 7.2.0 官方组件 | 19 / 19 文件 |
| L4 实例运行 | 真实部署 + 启动实例 + 路径断言 | Flowable + H2 内存引擎 | 6 / 6 场景 |

### 3.3 五个关键设计决策

| 决策 | 我们选了什么 | 被否决的方案 | 否决原因 |
| --- | --- | --- | --- |
| 大模型输出什么 | 只输出 IR（JSON） | 直接输出 BPMN XML | 命名空间、id 冲突、坐标缺失、标签闭合、转义五类随机故障，每次失败方式都不同，无法测试覆盖 |
| 依赖策略 | 核心链路零依赖 | 用 bpmn-moddle 做核心校验 | 引入 node_modules、版本锁定、供应链风险；评审克隆后卡在 npm install |
| 引擎选型 | Flowable 7 | Camunda 8 | Zeebe + Elasticsearch + gRPC，架构重，与“本机零污染”目标直接冲突 |
| 图形坐标 | 自己算 DI | 只生成逻辑段 | 引擎能跑但 bpmn-js 打开是白屏，演示当场翻车 |
| 自动修复边界 | 只修确定性错误 | 让 AI 猜缺失的连线 | 猜出来的流程无法解释，且会把错误固化进产物 |

### 3.4 三个核心技术难点

**难点一：模型幻觉导致的“看起来对、跑不通”。**

模型经常生成结构完整但语义非法的流程：多个结束事件、某审批节点没有出口、排他网关所有分支都带条件却没有默认流。最后一类最隐蔽——XML 合法、XSD 通过、引擎也能部署，但运行时所有条件都不满足，流程实例永久卡死。我们的做法是把这些问题全部前移到静态检查（src/graph.js 做可达性与死路分析），在进引擎之前就拦掉。

**难点二：图形坐标（DI）不能靠手写。**

BPMN 文件分两半：process 是逻辑，BPMNDiagram 是坐标。少了坐标段，引擎照跑，但可视化工具打开是空白页。我们实现了最长路径分层算法：按“从开始节点的最长距离”分层，同层纵向排开，正向连线走直角折线、回边绕行。选分层而非力导向，是因为力导向每次布局结果不同，会破坏“同输入同输出”的可复现性。

**难点三：修复必须克制。**

自动修复很容易做过头。我们划了一条硬线：缺 id、重复 id、自环、悬空端点、多余默认流可以修；缺节点类型、悬挂节点、不可达节点、网关缺默认流一律不修——因为这些需要业务判断，替他猜等于制造无法解释的流程。

## 四、应用场景

| 场景 | 典型描述 | 对应模板 |
| --- | --- | --- |
| 差旅报销 | 金额小于 X 主管批，超过 Y 财务复核 | amount_tiered（金额分级，阈值可参数化） |
| 请假审批 | 3 天以内主管批，超过 3 天走人事备案 | leave_days |
| 采购申请 | 按金额分三档：部门经理 / 采购部 / 财务总监 | purchase_request |
| 合同评审 | 法务、财务、业务三方并行会签后汇总 | contract_review（并行网关） |
| 入职办理 | 人事建档、IT 开户、行政工位三线并行 | onboarding_parallel（并行网关） |
| 用章申请 | 申请 → 审批 → 用印登记 | seal_apply |
| 费用报销 | 部门审核 → 财务审核 → 归档 | expense_reimburse |
| 通用单级 | 提交后一人审批即可 | simple_approval |

模板是纯 JSON 文件，业务方可以自行增删，不需要改代码——这让系统具备横向复制到新行业的能力。

# 五、创新点

### 5.1 让模型“只说选哪个模板”，而不是“画流程”

这是本作品与常见“让 AI 直接写 BPMN”方案的根本区别。模型被约束成只能输出一个极小的 JSON：

{"templateId":"amount_tiered","slots":{"tier1":2000,"tier2":8000},"confidence":0.95}

结构、连线、坐标、命名空间全部由确定性代码产生。这样“生成不出合法 XML”在物理上不可能发生，模型的自由度和系统可靠性之间的取舍被明确放在语义层，而不是语法层。

### 5.2 把“生成”降级为“选模板 + 填参数”

纯大模型自由生成的通过率不稳定，且每次结果不同。模板化后，稳定性来自模板本身，模型只承担它最擅长的小任务：理解一句话的意图、抽取阈值。实测 8 条业务描述端到端通过率 100%。

### 5.3 引擎规则反哺静态检查（真实案例）

项目在接入 Flowable 官方校验器后，发现自研图论检查漏了一条规则：

> *“Default sequenceflow has a condition, which is not allowed”*

含义是默认流不能同时带条件（默认流是“条件都不满足时的兜底”，自己再带条件就自相矛盾）。这条规则 XSD 查不出、标准模型库也查不出，只有引擎部署时才报。我们把这条规则补进了静态检查，并给修复器加了对应的确定性修复——现在这类错误在写文件之前就会被拦下。这说明多层验证不是重复劳动，而是互相补位。

### 5.4 把“环境零污染”做成可验证的工程原则

政务与企业客户对交付环境的干净程度很敏感。本项目把这件事从口号变成了 11 条可检查的工程原则，其中 10 条由脚本自动验证，红灯即不得交付。配套的环境守卫工具能给出基线与差异报告，实现“项目目录删掉即归零”。详见第七章。

# 六、验证与证据

**所有结论都可以在仓库内用一条命令复现：verify-all.cmd**

| 验证项 | 手段 | 结果 |
| --- | --- | --- |
| 单元与回归测试 | node test/run.js（正向 / 反向 / 可修复 / 自然语言 / 提示词 / 脚本卫生） | 128 项断言全部通过 |
| 大模型端到端 | run-llm-tests.cmd（真实调用 8 条业务描述） | 8 / 8 通过（100%） |
| BPMN 规范（L2） | OMG 官方 XSD + JDK 17 单文件校验器 | 19 / 19 文件通过 |
| BPMN 模型（L2b） | 第三方 bpmn-moddle | 19 / 19 文件通过 |
| 引擎接受（L4-lite） | Flowable 7.2.0 的 BpmnXMLConverter + ProcessValidator | 19 / 19 文件通过 |
| 实例运行（L4） | Flowable + H2 内存引擎：部署 → 启动实例 → 自动完成任务 → 路径断言 | 6 / 6 场景通过 |
| 确定性 | 同一 IR 连续序列化两次逐字节比对 | 完全一致，产物无时间戳 |

### 6.1 大模型实测样例

输入（真实测试用例）：

*“我要做一个差旅报销审批流，金额小于 2000 主管批，超过 8000 财务批”*

输出：

| 项 | 结果 |
| --- | --- |
| 选中模板 | amount_tiered（金额分级审批），置信度 0.95 |
| 模型理由 | 差旅报销按金额区间分级审批，2000 与 8000 为两个分级阈值 |
| 生成结果 | 7 个节点 / 8 条连线，5999 字节 |
| 条件表达式 | ${amount <= 2000} 与 ${amount > 2000 && amount <= 8000} |

| 项 | 结果 |
| --- | --- |
| 引擎校验 | Flowable 解析通过、流程校验通过 |
| 用量 | 输入 869 token / 输出 159 token |

### 6.2 实例运行实测记录

同一份 BPMN 文件、不同输入变量，实测走过的审批路径：

| 输入变量 | 实际路径 | 结论 |
| --- | --- | --- |
| amount = 800 | 提交报销单 → 主管审批 | 小额走主管，符合预期 |
| amount = 2500 | 提交报销单 → 总监审批 | 中额升级到总监，符合预期 |
| amount = 6000 | 提交报销单 → 财务复核 | 大额走财务，符合预期 |
| days = 2 | 填写请假单 → 直属主管审批 | 短期走主管 |
| days = 9 | 填写请假单 → 人事备案 | 长期走人事 |
| 合同会签 | 提交 → 法务/财务/业务三方并行 → 汇总结束 | 并行网关语义正确 |

这组结果说明：条件表达式不是文本装饰，它在真实引擎里驱动了不同的执行路径。

## 七、工程原则：把“不污染本机”变成可验证的约束

本项目的开发环境前提是：一台 Windows 家庭版机器，无法使用系统级沙盒或虚拟机。因此隔离只能做在用户态，原则也必须落到可检查的地方。

| 编号 | 原则 | 检查方式 |
| --- | --- | --- |
| P-01 | 磁盘边界：产物不出工作区 | 扫描源码中的系统路径引用 |
| P-02 | 分层零依赖：核心 0 依赖，前端依赖须登记 | 根依赖为 0 + web 依赖必须出现在第三方清单 |
| P-03 | 不碰系统层：无服务 / 注册表 / PATH / 管理员 | 扫描 reg add / sc create / setx 等 |
| P-04 | 进程边界：前台运行，不起常驻进程 | 扫描子进程调用 |
| P-05 | 密钥不落地：只从环境变量读 | 扫描硬编码密钥特征 |

| 编号 | 原则 | 检查方式 |
| --- | --- | --- |
| P-06 | 生成物确定性：同输入同字节 | 两次序列化比对字节 |
| P-07 | 模型不直接产出 XML | 校验管线顺序 + 拒绝非 IR 输入 |
| P-08 | 校验不可跳过，错误必须显式失败 | 坏图必须报错、好图必须零错 |
| P-09 | 可归零：产物不入库、可一键清理 | .gitignore 覆盖 + 项目内无残留产物 |
| P-10 | 诚实：未验证的不声称已验证 | 人工守则（本章 6.2 即其应用） |
| P-11 | 开源合规：许可证清晰、无传染性污染 | LICENSE / NOTICE / 第三方清单齐全，源码无 GPL |

实测数据：依赖安装前后，系统级 npm 缓存 802.5 MB 保持不变（零增长），全部依赖落在项目目录，可一键删除；安装环境守卫报告与基线完全一致。

# 八、交付物与使用方式

## 8.1 三条命令

| 命令 | 作用 |
| --- | --- |
| ai2bpmn.cmd "业务描述" | 调用大模型，一句话生成 BPMN |
| nl2bpmn.cmd "业务描述" | 离线模板匹配，不联网、不需要密钥 |
| demo.cmd | 启动本地 Web 应用（浏览器里输入、生成、下载） |
| verify-all.cmd | 五层验证一次跑完（测试 / XSD / moddle / 引擎） |

## 8.2 目录结构（节选）

| 路径 | 说明 |
| --- | --- |
| bin/ | 两个命令行入口：IR → BPMN、自然语言 → BPMN |
| src/ | 生成核心（零依赖）：契约、图论、布局、序列化、修复、提示词 |
| src/templates/ | 8 个审批流模板（JSON，可直接增删） |
| test/ | 正向 / 反向 / 可修复 / 自然语言用例，共 118 项断言 |

| 路径 | 说明 |
| --- | --- |
| tools/ | XSD 校验器、Flowable 引擎校验器（Java 单文件，无需构建工具） |
| web/ | 本地 Web 应用（生成 / 模板库 / 验证报告 / 设置）与第三方模型校验 |
| docs/ | 架构说明、第三方许可清单、本文档 |

### 8.3 开源合规声明

本项目由开发者主导架构设计与核心实现；部分辅助代码由大模型生成，经人工审查与重构，确保无 GPL / AGPL 传染性代码污染。

- 许可证：Apache License 2.0，仓库根目录附 LICENSE 与 NOTICE。
- 第三方组件：见 docs/THIRD-PARTY.md，核心链路运行时依赖为 0 个。
- 规范文件（OMG XSD）按需下载至缓存目录，不随仓库分发。
- 大模型密钥只从环境变量读取，不入库、不打印、不写入日志。

### 8.4 后续可扩展方向

- 模板库扩充：面向更多行业（项目立项、合同用印、值班调休等）沉淀模板。
- 多供应商支持：已抽象供应商层，可并行调用不同模型做投票或降级。
- 模板可视化编辑：在 Web 应用里直接编辑模板 JSON 并实时预览，进一步降低配置门槛。
- 表单与数据集成：把用户任务与真实表单、外部系统对接，形成可上线的审批中台。

—— 本文档所引数据均可通过仓库内 verify-all.cmd 复现 ——